ASSIGNMENT 1.2

Rule-Based Environmental Reasoner (Simple Reflex Agent)

Student	[Student Name]	Roll No	[Roll No]
Course	21CSE525T - Agent Technology	Project	Placement Preparation Agent

Unit 1 - Simple reflex architecture. File: assignments/a1_2/reflex_agent.py

1. Objective

Build a simple reflex agent that reads a stream of student performance percepts and reacts to each one with condition-action rules, without memory or planning.

2. Design

The agent is a **simple reflex agent**: the action depends only on the current percept. It keeps no beliefs and no goal. This is the weakest agent architecture in the Unit 1 taxonomy and is the baseline for later units.

The environment is a mock telemetry stream (sensor_stream) emitting 12 ticks. Each tick is one percept: topic, coding score, aptitude score, study hours, days to placement, and mock-interview rating.

parse_percept() turns raw telemetry into abstracted state predicates: weak_coding, weak_aptitude, idle, urgent, interview_unready and strong.

Six priority-ordered rules (R1 to R6) match those predicates. Every matching rule fires, and fired actions are printed in priority order. If no rule fires, the default action is to continue the current plan.

3. Rule base

R1 URGENT_REVISION: 7 days or fewer to placement -> rapid revision mode.

R2 CODING_REMEDIATION: coding score below 50 -> assign easy and medium problems on the topic.

R3 APTITUDE_DRILL: aptitude below 50 -> 30-question drill.

R4 MOCK_INTERVIEW: rating 4 or lower -> schedule a mock interview.

R5 IDLE_NUDGE: under 1 hour studied -> reminder notification.

R6 ADVANCE_LEVEL: coding and aptitude both 85 or higher -> unlock hard tier and Tier-1 companies.

4. Implementation

```
def parse_percept(raw):
    """Sensory parsing logic: normalise raw telemetry into agent state."""
    state = {
        "topic": raw["topic"],
        "coding": raw["coding_score"],
        "aptitude": raw["aptitude_score"],
        "hours": raw["study_hours_today"],
        "deadline": raw["days_to_placement"],
        "interview": raw["mock_interview_rating"],
    }
    # derived (abstracted) percepts
    state["weak_coding"] = state["coding"] &lt; 50
    state["weak_aptitude"] = state["aptitude"] &lt; 50
    state["idle"] = state["hours"] &lt; 1.0
    state["urgent"] = state["deadline"] &lt;= 7
    state["interview_unready"] = state["interview"] &lt;= 4
    state["strong"] = state["coding"] &gt;= 85 and state["aptitude"] &gt;= 85
    return state

class SimpleReflexAgent:
    def __init__(self, name="PlacementPrepAgent"):
        self.name = name
        self.log = []

    def act(self, raw_percept):
```

```
        state = parse_percept(raw_percept)
        fired = [(p, n, act(state)) for p, n, cond, act in RULES if cond(state)]
        fired.sort(key=lambda x: x[0])          # priority ordering

        header = (f"[{raw_percept['timestamp']}] TICK {raw_percept['tick']:02d} | "
                  f"topic={state['topic']:&lt;9} code={state['coding']:&gt;3}% "
                  f"apt={state['aptitude']:&gt;3}% hrs={state['hours']:&lt;4} "
                  f"D-{state['deadline']:&lt;2} mock={state['interview']}/10")
        print(header)
        self.log.append(header)

        if not fired:
            print("      -&gt; " + DEFAULT_ACTION)
            self.log.append("      -&gt; " + DEFAULT_ACTION)
        for pri, name, action in fired:
            line = f"      -&gt; [{name}] {action}"
            print(line)
            self.log.append(line)
        print()
        return fired
```

5. Result

The run processed 12 percepts and fired 18 rules in total (RUN SUMMARY line). The agent also writes its console output to execution_log.txt inside the a1_2 folder.

Execution output (log file)

Captured from **logs/1.2_reflex_agent.log** (python3 reflex_agent.py). Timestamps and some values differ between runs; this is the recorded run.

```
# Activity 1.2_reflex_agent
# command: python3 reflex_agent.py
# exit code: 0
# lines: 52

================================================================================
 PLACEMENT PREPARATION AGENT :: SIMPLE REFLEX ARCHITECTURE (Assignment 1.2)
================================================================================

[10:40:13] TICK 01 | topic=Graphs    code= 39% apt= 75% hrs=3.9  D-6  mock=9/10
      -&gt; [R1_URGENT_REVISION] ACTION: Switch to RAPID-REVISION mode (6 days left). Queue top-50 must-do problems.
      -&gt; [R2_CODING_REMEDIATION] ACTION: Assign 5 EASY + 2 MEDIUM problems on 'Graphs' (coding=39%).

[10:40:13] TICK 02 | topic=Arrays    code= 66% apt= 32% hrs=5.5  D-15 mock=1/10
      -&gt; [R3_APTITUDE_DRILL] ACTION: Push 30-question aptitude drill (aptitude=32%).
      -&gt; [R4_MOCK_INTERVIEW] ACTION: Schedule mock interview + communication feedback (rating=1/10).

[10:40:13] TICK 03 | topic=Arrays    code= 75% apt= 78% hrs=0.4  D-7  mock=9/10
      -&gt; [R1_URGENT_REVISION] ACTION: Switch to RAPID-REVISION mode (7 days left). Queue top-50 must-do problems.
      -&gt; [R5_IDLE_NUDGE] ACTION: Send reminder notification - only 0.4h studied today.

[10:40:13] TICK 04 | topic=OOPS      code= 27% apt= 97% hrs=0.7  D-16 mock=10/10
      -&gt; [R2_CODING_REMEDIATION] ACTION: Assign 5 EASY + 2 MEDIUM problems on 'OOPS' (coding=27%).
      -&gt; [R5_IDLE_NUDGE] ACTION: Send reminder notification - only 0.7h studied today.

[10:40:13] TICK 05 | topic=Arrays    code= 93% apt= 75% hrs=0.3  D-16 mock=1/10
      -&gt; [R4_MOCK_INTERVIEW] ACTION: Schedule mock interview + communication feedback (rating=1/10).
      -&gt; [R5_IDLE_NUDGE] ACTION: Send reminder notification - only 0.3h studied today.

[10:40:14] TICK 06 | topic=DBMS      code= 37% apt= 62% hrs=2.5  D-36 mock=2/10
      -&gt; [R2_CODING_REMEDIATION] ACTION: Assign 5 EASY + 2 MEDIUM problems on 'DBMS' (coding=37%).
      -&gt; [R4_MOCK_INTERVIEW] ACTION: Schedule mock interview + communication feedback (rating=2/10).

[10:40:14] TICK 07 | topic=DBMS      code= 59% apt= 96% hrs=4.9  D-13 mock=2/10
      -&gt; [R4_MOCK_INTERVIEW] ACTION: Schedule mock interview + communication feedback (rating=2/10).

[10:40:14] TICK 08 | topic=DBMS      code= 93% apt= 49% hrs=2.2  D-37 mock=2/10
      -&gt; [R3_APTITUDE_DRILL] ACTION: Push 30-question aptitude drill (aptitude=49%).
      -&gt; [R4_MOCK_INTERVIEW] ACTION: Schedule mock interview + communication feedback (rating=2/10).

[10:40:14] TICK 09 | topic=DBMS      code= 27% apt= 51% hrs=3.0  D-36 mock=7/10
      -&gt; [R2_CODING_REMEDIATION] ACTION: Assign 5 EASY + 2 MEDIUM problems on 'DBMS' (coding=27%).

[10:40:14] TICK 10 | topic=OS        code= 60% apt= 84% hrs=3.5  D-31 mock=6/10
      -&gt; ACTION: Continue current study plan (no rule fired).

[10:40:14] TICK 11 | topic=Graphs    code= 51% apt= 48% hrs=4.2  D-17 mock=2/10
```

```
      -&gt; [R3_APTITUDE_DRILL] ACTION: Push 30-question aptitude drill (aptitude=48%).
      -&gt; [R4_MOCK_INTERVIEW] ACTION: Schedule mock interview + communication feedback (rating=2/10).

[10:40:14] TICK 12 | topic=DBMS      code= 58% apt= 92% hrs=3.0  D-23 mock=8/10
      -&gt; ACTION: Continue current study plan (no rule fired).

================================================================================
 RUN SUMMARY: 12 percepts processed | 18 condition-action rules fired
================================================================================
 Console log written to execution_log.txt
```

6. Limitations and viva points

No memory: the same percept always produces the same action, so the agent cannot notice a trend across ticks.

Thresholds (50, 85, 7 days) are hand-chosen, not learned.

This is why Unit 4 adds a belief base and goals.

7. Conclusion

The reflex agent shows that a fixed mapping from percept to action can already give useful, reactive behaviour, and it sets up the limits that the later units address.